CS462 – Big Data Systems

Lab 0: Docker Setup & Container Environment

Bellevue College – Spring 2026

Course	CS462 – Big Data Systems
Lab Number	Lab 0
Topics	Docker Desktop, Docker Compose, Container Images, Python Environment
Estimated Time	1–2 hours
Points	100
Prerequisite	None – this lab assumes no prior Docker experience
Submission	Canvas – upload a single ZIP file (see Deliverables section)

Learning Objectives

By the end of this lab you will be able to:

- Install and configure Docker Desktop on macOS, Windows, or Linux
- Allocate appropriate system resources (memory) to Docker
- Write a Dockerfile and a docker-compose.yml to define a reproducible Python environment
- Build a custom Docker image and launch an interactive container shell
- Verify that Python, PyArrow, and Pandas are correctly installed inside the container
- Run a short Python validation script to confirm the environment is ready for data work

Prerequisites

- A computer running macOS (Intel or Apple Silicon), Windows 10/11or Linux
- At least 4 GB of free RAM available for Docker
- An internet connection for downloading Docker Desktop and the base Python image
- Basic familiarity with navigating the command line (cd, ls / dir)

No prior Docker experience is required.

Environment Setup

Section A – macOS Setup

Step 1 — Install Docker Desktop

Go to https://www.docker.com/products/docker-desktop/ and download the correct installer for your chip:

- Intel Mac: choose “Mac with Intel Chip”
- Apple Silicon (M1/M2/M3/M4): choose “Mac with Apple Silicon”

Run the .dmg installer, drag Docker to Applications, and launch Docker Desktop. It will ask for your system password once to install a privileged helper — this is expected.

Step 2 — Allocate Memory

Open Docker Desktop → Settings (gear icon) → Resources → Memory and raise the slider to at least 4 GB. Click “Apply & Restart.”

Step 3 — Enable Rosetta (Apple Silicon only)

Note: Apple Silicon (M1/M2/M3/M4): The Python base image used in this lab is built for linux/amd64. Docker Desktop emulates this via Rosetta 2 translation. Go to Settings → General and check “Use Rosetta for x86_64/amd64 emulation on Apple Silicon.” Without this, the build step may fail with a platform mismatch error.

Step 4 — Verify

Open Terminal and run both commands below. Both should print version numbers — no errors.

```
docker --version
docker compose version
```

Section B – Windows Setup

Step 1 — Enable WSL 2

Docker Desktop on Windows requires the Windows Subsystem for Linux version 2 (WSL 2). Open PowerShell as Administrator and run:

```
wsl --install
```

This installs WSL 2 and Ubuntu in one step. Restart your computer when prompted.

Note: If WSL is already present but you are unsure of the version, run wsl --set-default-version 2 to ensure version 2 is the default.

Step 2 — Install Docker Desktop

Go to https://www.docker.com/products/docker-desktop/ and download the Windows (AMD64) installer. During setup, confirm that “Use WSL 2 instead of Hyper-V” is checked.

Step 3 — Allocate Memory

On most Windows systems, Docker automatically allocates more than enough memory. You can check by running the following command:

```
docker info | findstr /i memory
```

If Docker reports less than 4GB of memory on your machine, please use Step 4 to manually adjust memory allocation.

Step 4 — (Optional) Tune WSL 2 Memory

If Docker Desktop reports very little available memory, WSL 2 may be drawing from a limited pool. Create or edit C:\Users\<YourName>\.wslconfig:

```
[wsl2]
memory=8GB
processors=4
```

Then run wsl --shutdown in PowerShell and restart Docker Desktop.

Step 5 — Verify

Open PowerShell or Windows Terminal and run:

```
docker --version
docker compose version
```

Both should print version numbers with no errors.

Section C – Linux Setup

Step 1 — Install Docker Engine

On most Linux distributions, install Docker Engine directly from Docker’s official repository. For Ubuntu/Debian, the easiest path is the convenience script: https://docs.docker.com/engine/install/. Run:

```
curl -fsSL https://get.docker.com -o get-docker.sh && sudo sh get-docker.sh
```

For Fedora, RHEL, or other distros, follow the distribution-specific instructions on the Docker docs page above.

Step 2 — Add Your User to the docker Group

So you can run Docker without sudo, add your user to the docker group and start a new shell session:

```
sudo usermod -aG docker $USER && newgrp docker
```

Step 3 — Enable and Start the Docker Service

```
sudo systemctl enable --now docker
```

Step 4 — Install Docker Compose Plugin

The convenience script installs the Compose v2 plugin automatically. If yours did not, install it via your package manager (e.g., `sudo apt-get install docker-compose-plugin`).

Step 5 — Verify

Open a terminal and run:

```
docker --version
docker compose version
```

Both should print version numbers with no errors. Then run `docker run hello-world` to confirm the daemon is working.

Note on Memory: On Linux, Docker runs natively and uses host memory directly — there is no separate VM to configure. Make sure your machine has at least 4 GB of RAM available for containers.

Part 1: Verify the Docker Installation (30 points)

Before building anything, confirm that Docker Desktop is running correctly and that your memory settings are in place.

1.1 Confirm Docker is Running

Open a terminal (macOS) or PowerShell (Windows) and run:

```
docker --version
docker compose version
docker info | grep -i memory
```

The first two commands should each print a version string. The third command prints Docker's view of available memory — confirm it shows at least 4 GiB.

Note: On Windows, use docker info | findstr /i memory instead of grep.

Minimum versions: You should at a minimum be on Docker Engine `29.3.1` or newer, and Docker Compose `v5.1.1` or newer. Older releases may work but may be impossible to troubleshoot or may simply not work for this course.

Take a screenshot of all three commands and their output in one terminal window. Label it screenshot-1a.png.

1.2 Confirm Memory Allocation in Docker Desktop

Open Docker Desktop, go to Settings → Resources, and take a screenshot showing the Memory slider set to 4 GB or higher. Label it screenshot-1b.png.

Part 1 – Deliverables

- screenshot-1a.png – terminal showing docker --version, docker compose version, and memory output
- screenshot-1b.png – Docker Desktop Settings showing memory ≥ 4 GB

Part 2: Build and Connect to the Container (40 points)

In this part you will create the two files that define your lab container, build the image, and connect to it.

2.1 Create the Lab Folder and Files

Create a new folder called lab0 on your computer, and inside it create an empty subfolder called data. Then create the two files below inside lab0.

File 1 – Dockerfile

This instructs Docker to start from an official Python 3.11 image, set a working directory, and install PyArrow and Pandas.

Dockerfile

```
FROM python:3.11-slim
WORKDIR /lab
RUN pip install --no-cache-dir pyarrow==17.0.0 pandas==2.2.3
CMD ["bash"]
```

File 2 – docker-compose.yml

This defines the service, mounts your local data/ folder into the container, and keeps a terminal open.

docker-compose.yml

```
version: "3"
services:
  lab0:
    build: .
    image: cs462-lab0:latest
    container_name: lab0
    volumes:
      - ./data:/lab/data
    stdin_open: true
    tty: true
    command: bash
```

Note: Make sure that all the indentations are using spaces and are the way they are shown here. Otherwise you might get a cryptic parsing error. Some editors with auto-formatting may convert the spaces into tabs which can cause these errors.

2.2 Build the Image

Open a terminal, navigate to your lab0 folder, and run:

```
docker compose build
```

Docker will download the python:3.11-slim base image and install the two packages. This takes 2–5 minutes on first run. A successful build ends with a line similar to:

```
 => exporting to image                                          0.1s
```

Note: Apple Silicon Mac: If you see a “WARNING: The requested image’s platform (linux/amd64) does not match” message, add platform: linux/amd64 on a new line directly below build: . in docker-compose.yml and run docker compose build again.

Take a screenshot of the completed build output. Label it screenshot-2a.png.

2.3 Enter the Container and Verify the Packages

Launch an interactive shell inside the container:

```
docker compose run --rm lab0 bash
```

Note: If you are getting errors mounting the 'data' directory during this command, check lower/upper case for your complete path. MacOS is not case sensitive, but Docker is. Oftentimes, your 'pwd' in your terminal is '/users/', but should be '/Users/' for Docker purposes. In that case, 'cd' into '/Users/' and check if that resolves the issue.

Your prompt changes to root@<id>:/lab#, confirming you are inside the container. Now verify that both packages are installed:

```
python3 -c "import pyarrow; print('PyArrow', pyarrow.__version__)"
python3 -c "import pandas;  print('Pandas',  pandas.__version__)"
```

Both commands should print a version number with no ImportError. Take a screenshot of the prompt change and both version outputs. Label it screenshot-2b.png.

Part 2 – Deliverables

- screenshot-2a.png – completed docker compose build output
- screenshot-2b.png – container prompt + PyArrow and Pandas version outputs

Part 3: Environment Validation Script (30 points)

To confirm the entire environment is working end-to-end, you will run a short Python script that creates a dataset, loads it into a Pandas DataFrame, and converts it to a PyArrow Table. This exercise also introduces the relationship between these two libraries, which you will use extensively in Lab 3.

3.1 Create and Run the Validation Script

Inside the container, create the following script at /lab/data/validate.py:

validate.py

```
import pyarrow as pa
import pandas as pd

# 1. Create a small dataset as a Python list of dicts
records = [
    {"name": "Alice",   "score": 92, "grade": "A"},
    {"name": "Bob",     "score": 78, "grade": "C"},
    {"name": "Charlie", "score": 85, "grade": "B"},
    {"name": "Diana",   "score": 96, "grade": "A"},
    {"name": "Ethan",   "score": 71, "grade": "C"},
]

# 2. Load into a Pandas DataFrame
df = pd.DataFrame(records)
print("=== Pandas DataFrame ===")
print(df)
print(f"\nShape : {df.shape}  (rows, columns)")
print(f"dtypes:\n{df.dtypes}")

# 3. Convert the DataFrame to a PyArrow Table
table = pa.Table.from_pandas(df)
print("\n=== PyArrow Table ===")
print(f"Schema : {table.schema}")
print(f"Rows   : {table.num_rows}")
print(f"Columns: {table.num_columns}")

print("\nEnvironment OK — ready for Lab 3.")
```

Run the script:

```
python3 /lab/data/validate.py
```

The final line should print “Environment OK — ready for Lab 3.” Take a screenshot of the full output. Label it screenshot-3a.png.

Part 3 – Deliverables

- screenshot-3a.png – full output of validate.py including the DataFrame, schema, and final confirmation line

Tearing Down the Container

Type exit (or press Ctrl-D) to leave the container shell. Because you used --rm, the container is removed automatically. Your lab0 folder and the data/ subfolder remain on your computer and will be reused in Lab 1.

Do not delete the lab0 folder or the Docker image — Lab 1 depends on them.

Submission Instructions

Create a ZIP file named lab0_<YourLastName>_<YourFirstName>.zip containing:

- A write-up document (PDF or DOCX) that includes:
- Your name, student ID, and the date
- A brief description (2–3 sentences) of what you did in each Part
- Answers to the reflection questions below
- All screenshots labeled as described (screenshot-1a through screenshot-3a)
- Your Dockerfile, docker-compose.yml, and validate.py

Reflection Questions

Answer these questions in your write-up (2–4 sentences each):

- What is Docker and what problem does it solve for a class like CS462? How does running Python inside a container ensure consistency across students' machines, even though some are on macOS and others on Windows or Linux?
- What is the difference between a Docker image and a Docker container? When you ran docker compose build, what was created? When you ran docker compose run, what happened next?
- The Dockerfile installs specific package versions (pyarrow==17.0.0, pandas==2.2.3). Why is pinning exact versions important in a reproducible environment? What could go wrong if you installed the latest version instead?

Grading Rubric

Task / Deliverable	Points	Grading Criteria
Part 1 – Docker version & compose version confirmed	15	Screenshot shows both commands printing version strings with no errors
Part 1 – Memory allocation confirmed	15	Docker Desktop screenshot shows memory slider set to ≥ 4 GB

Task / Deliverable	Points	Grading Criteria
Part 2 – Image built successfully	20	Screenshot shows docker compose build completing without errors
Part 2 – PyArrow and Pandas verified in container	20	Both import + print commands run inside the container with no ImportError
Part 3 – Validation script runs end-to-end	30	validate.py output shows correct DataFrame shape, Arrow schema, and final confirmation line
Reflection Questions	30	3 questions × 10 pts each; answers are 2–4 sentences and demonstrate understanding
TOTAL	**130**	

Note: The due date for this lab is posted on Canvas. Submissions received after that date will receive reduced points in accordance with the course late-work policy.